

Netflix-Like Streaming Platform Database Architecture

Table of Contents

1. [Overview](#)
 2. [Requirements](#)
 3. [Capacity Estimation](#)
 4. [Schema Design](#)
 5. [ASCII ER Diagram](#)
 6. [Indexing Strategy](#)
 7. [Query Patterns](#)
 8. [Scaling Strategy](#)
 9. [Tradeoffs](#)
 10. [Interview Discussion Points](#)
 11. [Common Interview Follow-ups](#)
 12. [Performance Considerations](#)
 13. [Cross-References](#)
-

Overview

A streaming platform database must handle an extreme read-to-write ratio. Millions of concurrent users browse, start playback, and pause — generating constant reads against a catalog that changes slowly. The schema presented here separates content metadata (write-rarely, read-often) from user activity data (write-frequently) and recommendation data (compute-intensive, cache-aggressively).

Requirements

Functional Requirements

- Users subscribe with monthly/annual plans
- Multiple profiles per account (like Netflix Kids)
- Content library: movies, TV series, episodes, documentaries
- Licensing: each title available per region, expiry managed
- Watch history: resume playback from where you left off
- Personalized recommendations
- Download for offline viewing
- Search titles by name, genre, actor, director
- Billing: recurring charges, plan upgrades/downgrades, invoices

Non-Functional Requirements

- 250M+ subscribers globally
 - 5 profiles per account average
 - 200M+ hours of content watched per day
 - 15K+ titles in catalog at any time
 - 99.99% availability for playback APIs
 - Sub-50ms latency for "Continue Watching" and homepage personalization
 - Data retained for 5 years for personalization, 7 years for billing
-

Capacity Estimation

Metric	Estimate
Subscribers	250M
Profiles (active)	1.25B
Daily active users	100M
Watch events per day	200M
New titles added per month	200
Recommendation rows per profile	50
Billing events per month	250M

Storage Estimates

Table	Rows	Row Size	Total
accounts	250M	600B	150GB
profiles	1.25B	300B	375GB
titles	15K	4KB	60MB
episodes	500K	1KB	500MB
watch_history	50B (5yr)	200B	10TB
recommendations	62.5B	100B	6.25TB
licenses	5M	300B	1.5GB
billing_invoices	1.5B	500B	750GB

Annual write volume (watch_history): ~73B rows/yr → ~14.6TB/yr
Peak QPS (watch events): ~2,300 writes/sec sustained, ~10K/sec at peak
Peak QPS (reads): ~100K reads/sec at evening prime time

Schema Design

```
-- =====
-- EXTENSIONS
-- =====

CREATE EXTENSION IF NOT EXISTS "uuid-osspl";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";
CREATE EXTENSION IF NOT EXISTS "unaccent";

-- =====
-- ENUM TYPES
```

```

-- =====
CREATE TYPE subscription_plan AS ENUM ('basic', 'standard', 'premium', 'family');
CREATE TYPE subscription_status AS ENUM ('active', 'paused', 'cancelled',
'past_due');
CREATE TYPE content_type AS ENUM ('movie', 'series', 'documentary', 'short',
'special');
CREATE TYPE video_quality AS ENUM ('SD', 'HD', 'FHD', 'UHD');
CREATE TYPE billing_status AS ENUM ('pending', 'paid', 'failed', 'refunded',
'void');
CREATE TYPE download_status AS ENUM ('queued', 'downloading', 'ready', 'expired',
'error');
CREATE TYPE rating_system AS ENUM ('G', 'PG', 'PG-13', 'R', 'NC-17', 'TV-Y', 'TV-G',
'TV-PG', 'TV-14', 'TV-MA');

-- =====
-- ACCOUNTS AND SUBSCRIPTIONS
-- =====

CREATE TABLE accounts (
    account_id      UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    email           VARCHAR(320)  NOT NULL,
    password_hash   VARCHAR(255)  NOT NULL,
    plan            subscription_plan NOT NULL DEFAULT 'standard',
    status          subscription_status NOT NULL DEFAULT 'active',
    country_code    CHAR(2)       NOT NULL,
    locale          VARCHAR(10)   NOT NULL DEFAULT 'en-US',
    timezone        VARCHAR(60)   NOT NULL DEFAULT 'UTC',
    created_at      TIMESTAMPTZ   NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ   NOT NULL DEFAULT now(),
    trial_ends_at   TIMESTAMPTZ,
    CONSTRAINT uq_accounts_email UNIQUE (email)
);

CREATE TABLE profiles (
    profile_id      UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    account_id      UUID          NOT NULL REFERENCES accounts(account_id) ON
DELETE CASCADE,
    name            VARCHAR(100)  NOT NULL,
    avatar_url      VARCHAR(500),
    is_kids         BOOLEAN       NOT NULL DEFAULT FALSE,
    max_rating      rating_system NOT NULL DEFAULT 'TV-MA',
    language        VARCHAR(10)  NOT NULL DEFAULT 'en',
    autoplay_next    BOOLEAN       NOT NULL DEFAULT TRUE,
    sort_order      SMALLINT      NOT NULL DEFAULT 0,
    created_at      TIMESTAMPTZ   NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ   NOT NULL DEFAULT now(),
    CONSTRAINT uq_profile_name_per_account UNIQUE (account_id, name),
    CONSTRAINT chk_profiles_per_account CHECK (sort_order BETWEEN 0 AND 4)
);

-- Plan definition table (drives feature gating)
CREATE TABLE subscription_plans (
    plan            subscription_plan PRIMARY KEY,

```

```

max_profiles    SMALLINT    NOT NULL,
max_streams     SMALLINT    NOT NULL,
max_downloads   SMALLINT    NOT NULL,
max_quality     video_quality NOT NULL,
price_usd_cents INT         NOT NULL,
description      TEXT
);

INSERT INTO subscription_plans VALUES
('basic',    1, 1, 1, 'SD', 899, 'Basic plan'),
('standard', 2, 2, 2, 'FHD', 1549, 'Standard HD plan'),
('premium',  4, 4, 4, 'UHD', 2299, 'Premium 4K plan'),
('family',   5, 4, 6, 'FHD', 1999, 'Family plan');

-- =====
-- BILLING
-- =====

CREATE TABLE payment_methods (
    pm_id          UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    account_id     UUID          NOT NULL REFERENCES accounts(account_id),
    provider        VARCHAR(50)   NOT NULL,
    token           VARCHAR(500)  NOT NULL,
    last_four       CHAR(4),
    expiry_month    SMALLINT,
    expiry_year     SMALLINT,
    is_default      BOOLEAN       NOT NULL DEFAULT FALSE,
    created_at      TIMESTAMPTZ   NOT NULL DEFAULT now()
);

CREATE TABLE billing_periods (
    period_id      UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    account_id     UUID          NOT NULL REFERENCES accounts(account_id),
    plan            subscription_plan NOT NULL,
    period_start    DATE          NOT NULL,
    period_end      DATE          NOT NULL,
    amount_cents    INT           NOT NULL,
    currency        CHAR(3)       NOT NULL DEFAULT 'USD',
    status          billing_status NOT NULL DEFAULT 'pending',
    pm_id           UUID          REFERENCES payment_methods(pm_id),
    provider_charge_id VARCHAR(200),
    charged_at      TIMESTAMPTZ,
    failed_at       TIMESTAMPTZ,
    failure_reason   TEXT,
    created_at      TIMESTAMPTZ   NOT NULL DEFAULT now(),
    CONSTRAINT uq_billing_period UNIQUE (account_id, period_start)
) PARTITION BY RANGE (period_start);

CREATE TABLE billing_periods_2024 PARTITION OF billing_periods
    FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
CREATE TABLE billing_periods_2025 PARTITION OF billing_periods
    FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
CREATE TABLE billing_periods_2026 PARTITION OF billing_periods

```

```

FOR VALUES FROM ('2026-01-01') TO ('2027-01-01');

-- =====
-- CONTENT CATALOG
-- =====

CREATE TABLE genres (
  genre_id      SERIAL          PRIMARY KEY,
  name          VARCHAR(100)    NOT NULL,
  slug          VARCHAR(100)    NOT NULL,
  CONSTRAINT uq_genres_slug UNIQUE (slug)
);

CREATE TABLE people (
  person_id     UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
  name          VARCHAR(200)    NOT NULL,
  birth_date    DATE,
  bio           TEXT,
  photo_url     VARCHAR(500),
  search_vector TSVECTOR
);

CREATE TABLE titles (
  title_id      UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
  content_type  content_type    NOT NULL,
  name          VARCHAR(500)    NOT NULL,
  original_name VARCHAR(500),
  slug          VARCHAR(500)    NOT NULL,
  synopsis      TEXT,
  tagline       VARCHAR(500),
  release_year  SMALLINT,
  end_year      SMALLINT,      -- for series
  rating        rating_system,
  runtime_minutes INT,         -- for movies
  poster_url    VARCHAR(1000),
  backdrop_url  VARCHAR(1000),
  trailer_url   VARCHAR(1000),
  imdb_id       VARCHAR(20),
  is_original   BOOLEAN        NOT NULL DEFAULT FALSE, -- Netflix Original
  is_active     BOOLEAN        NOT NULL DEFAULT TRUE,
  avg_user_rating NUMERIC(3,2),
  total_ratings INT            NOT NULL DEFAULT 0,
  search_vector TSVECTOR,
  created_at    TIMESTAMPTZ    NOT NULL DEFAULT now(),
  updated_at    TIMESTAMPTZ    NOT NULL DEFAULT now(),
  CONSTRAINT uq_titles_slug UNIQUE (slug)
);

CREATE TABLE title_genres (
  title_id      UUID            NOT NULL REFERENCES titles(title_id) ON DELETE
CASCADE,
  genre_id      INT             NOT NULL REFERENCES genres(genre_id),
  PRIMARY KEY (title_id, genre_id)

```

```

);

CREATE TABLE title_cast (
    title_id        UUID           NOT NULL REFERENCES titles(title_id) ON DELETE
    CASCADE,
    person_id       UUID           NOT NULL REFERENCES people(person_id),
    role            VARCHAR(50)    NOT NULL,    -- 'actor', 'director', 'writer',
    'producer'
    character_name  VARCHAR(300),
    sort_order      SMALLINT       NOT NULL DEFAULT 0,
    PRIMARY KEY (title_id, person_id, role)
);

CREATE TABLE seasons (
    season_id       UUID           PRIMARY KEY DEFAULT uuid_generate_v4(),
    title_id        UUID           NOT NULL REFERENCES titles(title_id) ON DELETE
    CASCADE,
    season_number   SMALLINT       NOT NULL,
    name            VARCHAR(200),
    synopsis        TEXT,
    release_year    SMALLINT,
    episode_count   SMALLINT       NOT NULL DEFAULT 0,
    CONSTRAINT uq_seasons UNIQUE (title_id, season_number)
);

CREATE TABLE episodes (
    episode_id      UUID           PRIMARY KEY DEFAULT uuid_generate_v4(),
    season_id       UUID           NOT NULL REFERENCES seasons(season_id) ON DELETE
    CASCADE,
    title_id        UUID           NOT NULL REFERENCES titles(title_id),
    episode_number  SMALLINT       NOT NULL,
    name            VARCHAR(300)   NOT NULL,
    synopsis        TEXT,
    runtime_minutes INT           NOT NULL,
    still_url       VARCHAR(1000),
    is_active       BOOLEAN        NOT NULL DEFAULT TRUE,
    CONSTRAINT uq_episodes UNIQUE (season_id, episode_number)
);

-- =====
-- LICENSING (Content Rights by Region)
-- =====

CREATE TABLE licenses (
    license_id      UUID           PRIMARY KEY DEFAULT uuid_generate_v4(),
    title_id        UUID           NOT NULL REFERENCES titles(title_id),
    country_code    CHAR(2)        NOT NULL,
    valid_from      DATE           NOT NULL,
    valid_until     DATE,          -- NULL = perpetual
    is_exclusive    BOOLEAN        NOT NULL DEFAULT FALSE,
    notes           TEXT,
    created_at      TIMESTAMPTZ    NOT NULL DEFAULT now(),
    CONSTRAINT uq_license UNIQUE (title_id, country_code, valid_from)
);

```

```

);

-- Fast availability check view
CREATE MATERIALIZED VIEW title_availability AS
SELECT DISTINCT
    l.title_id,
    l.country_code,
    TRUE AS available
FROM licenses l
WHERE l.valid_from <= CURRENT_DATE
    AND (l.valid_until IS NULL OR l.valid_until > CURRENT_DATE);

CREATE UNIQUE INDEX ON title_availability (title_id, country_code);

-- =====
-- VIDEO ASSETS
-- =====
CREATE TABLE video_assets (
    asset_id        UUID                PRIMARY KEY DEFAULT uuid_generate_v4(),
    episode_id      UUID                REFERENCES episodes(episode_id),
    title_id        UUID                REFERENCES titles(title_id),    -- for movies
    quality         video_quality      NOT NULL,
    codec           VARCHAR(20)         NOT NULL DEFAULT 'h264',
    cdn_url         VARCHAR(1000)       NOT NULL,
    file_size_bytes BIGINT,
    duration_seconds INT                NOT NULL,
    bitrate_kbps    INT,
    language        VARCHAR(10)        NOT NULL DEFAULT 'en',
    is_active       BOOLEAN             NOT NULL DEFAULT TRUE,
    CONSTRAINT chk_asset_owner CHECK (
        (episode_id IS NOT NULL) != (title_id IS NOT NULL)
    )
);

CREATE TABLE subtitles (
    subtitle_id     UUID                PRIMARY KEY DEFAULT uuid_generate_v4(),
    asset_id        UUID                NOT NULL REFERENCES video_assets(asset_id),
    language        VARCHAR(10)        NOT NULL,
    is_forced       BOOLEAN             NOT NULL DEFAULT FALSE,
    url             VARCHAR(1000)       NOT NULL
);

-- =====
-- WATCH HISTORY (Very High Volume)
-- =====
CREATE TABLE watch_history (
    watch_id        UUID                PRIMARY KEY DEFAULT uuid_generate_v4(),
    profile_id      UUID                NOT NULL REFERENCES profiles(profile_id),
    title_id        UUID                NOT NULL REFERENCES titles(title_id),
    episode_id      UUID                REFERENCES episodes(episode_id),
    started_at      TIMESTAMPTZ        NOT NULL DEFAULT now(),
    last_watched_at TIMESTAMPTZ        NOT NULL DEFAULT now(),

```

```

    progress_seconds INT          NOT NULL DEFAULT 0,
    duration_seconds INT          NOT NULL,
    completed        BOOLEAN      NOT NULL DEFAULT FALSE,
    device_type      VARCHAR(50),
    quality          video_quality NOT NULL DEFAULT 'HD'
) PARTITION BY RANGE (started_at);

-- Monthly partitions
CREATE TABLE watch_history_2024_01 PARTITION OF watch_history
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');
-- ... (automate with pg_partman)

-- For "Continue Watching" – latest progress per title per profile
-- This is a separate hot table to avoid partition scan
CREATE TABLE watch_progress (
    profile_id      UUID          NOT NULL REFERENCES profiles(profile_id),
    title_id        UUID          NOT NULL REFERENCES titles(title_id),
    episode_id      UUID          REFERENCES episodes(episode_id),
    progress_seconds INT          NOT NULL DEFAULT 0,
    duration_seconds INT          NOT NULL,
    completed        BOOLEAN      NOT NULL DEFAULT FALSE,
    updated_at      TIMESTAMPTZ   NOT NULL DEFAULT now(),
    PRIMARY KEY (profile_id, title_id)
);

-- =====
-- USER RATINGS
-- =====
CREATE TABLE user_ratings (
    rating_id      UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    profile_id      UUID          NOT NULL REFERENCES profiles(profile_id),
    title_id        UUID          NOT NULL REFERENCES titles(title_id),
    rating          SMALLINT      NOT NULL,    -- 1-5 stars
    created_at      TIMESTAMPTZ   NOT NULL DEFAULT now(),
    CONSTRAINT uq_user_rating UNIQUE (profile_id, title_id),
    CONSTRAINT chk_rating_range CHECK (rating BETWEEN 1 AND 5)
);

-- =====
-- RECOMMENDATIONS
-- =====
CREATE TABLE recommendations (
    profile_id      UUID          NOT NULL REFERENCES profiles(profile_id),
    title_id        UUID          NOT NULL REFERENCES titles(title_id),
    score           FLOAT         NOT NULL,
    reason          VARCHAR(100), -- 'because_you_watched', 'trending',
'similar_users'
    row_number      SMALLINT      NOT NULL,
    generated_at    TIMESTAMPTZ   NOT NULL DEFAULT now(),
    PRIMARY KEY (profile_id, title_id)
);

```

```

CREATE INDEX idx_recommendations_profile ON recommendations (profile_id, score
DESC);

-- Refresh by ML pipeline every few hours; use TRUNCATE + bulk insert
-- Never query this table without profile_id in the WHERE clause

-- =====
-- DOWNLOADS (Offline Viewing)
-- =====

CREATE TABLE downloads (
  download_id      UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
  profile_id       UUID          NOT NULL REFERENCES profiles(profile_id),
  asset_id         UUID          NOT NULL REFERENCES video_assets(asset_id),
  title_id         UUID          NOT NULL REFERENCES titles(title_id),
  episode_id       UUID          REFERENCES episodes(episode_id),
  device_id        VARCHAR(200)  NOT NULL,
  status           download_status NOT NULL DEFAULT 'queued',
  expires_at       TIMESTAMPTZ   NOT NULL,
  created_at       TIMESTAMPTZ   NOT NULL DEFAULT now(),
  CONSTRAINT uq_download UNIQUE (profile_id, asset_id, device_id)
);

-- =====
-- SEARCH INDEX MAINTENANCE
-- =====

CREATE OR REPLACE FUNCTION update_title_search_vector()
RETURNS TRIGGER AS $$
BEGIN
  NEW.search_vector :=
    setweight(to_tsvector('english', COALESCE(NEW.name, '')), 'A') ||
    setweight(to_tsvector('english', COALESCE(NEW.original_name, '')), 'B') ||
    setweight(to_tsvector('english', COALESCE(NEW.synopsis, '')), 'C') ||
    setweight(to_tsvector('english', COALESCE(NEW.tagline, '')), 'D');
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_titles_search
  BEFORE INSERT OR UPDATE OF name, original_name, synopsis, tagline
  ON titles
  FOR EACH ROW EXECUTE FUNCTION update_title_search_vector();

```

ASCII ER Diagram

```

+-----+      +-----+      +-----+
| accounts |1---N| profiles |1---N| watch_progress |
+-----+      +-----+      +-----+
      |1              |1
      |N              |N
+-----+      +-----+      +-----+

```

```

| billing |      |watch_history|      |user_ratings|
| periods |      +-----+      +-----+
+-----+

      +-----+
      | titles |1---N| seasons |1---N| episodes |
      +-----+      +-----+      +-----+
      |1
      |N
      +-----+
      | genres |
      | (via   |
      |title_gen)|
      +-----+
      +-----+
      | people |
      | (via   |
      |title_cast|
      +-----+

      +-----+
      | video_assets|
      +-----+
      |1
      |N
      +-----+
      | subtitles |
      +-----+

+-----+      +-----+
| titles |1---N| licenses |
+-----+      +-----+

+-----+      +-----+
| profiles |1---N| recommendations |
+-----+      +-----+

+-----+      +-----+
| profiles |1---N| downloads |
+-----+      +-----+

```

Indexing Strategy

```

-- Accounts: authentication
CREATE INDEX idx_accounts_email      ON accounts (lower(email));
CREATE INDEX idx_accounts_status     ON accounts (status) WHERE status = 'active';

-- Profiles: account lookup and kids filter
CREATE INDEX idx_profiles_account    ON profiles (account_id);

-- Titles: browse and search
CREATE INDEX idx_titles_type         ON titles (content_type, is_active);
CREATE INDEX idx_titles_search       ON titles USING GIN (search_vector);
CREATE INDEX idx_titles_rating       ON titles (avg_user_rating DESC NULLS LAST)
    WHERE is_active = TRUE;
CREATE INDEX idx_titles_year         ON titles (release_year DESC);

-- Genres: M:N join table
CREATE INDEX idx_title_genres_genre  ON title_genres (genre_id);

```

```

-- People search
CREATE INDEX idx_people_search      ON people USING GIN (search_vector);

-- Episodes
CREATE INDEX idx_episodes_season    ON episodes (season_id, episode_number);
CREATE INDEX idx_episodes_title     ON episodes (title_id);

-- Licenses: availability check (country + date)
CREATE INDEX idx_licenses_country   ON licenses (country_code, valid_from,
valid_until);
CREATE INDEX idx_licenses_title     ON licenses (title_id);

-- Watch progress: homepage "Continue Watching" (hot path)
CREATE INDEX idx_watch_progress_profile ON watch_progress (profile_id, updated_at
DESC)
    WHERE completed = FALSE;

-- Watch history: analytics and "watched before" check
CREATE INDEX idx_watch_history_profile ON watch_history (profile_id, started_at
DESC);

-- Billing: account invoice history
CREATE INDEX idx_billing_account     ON billing_periods (account_id, period_start
DESC);
CREATE INDEX idx_billing_status      ON billing_periods (status, period_start)
    WHERE status = 'failed';

-- Recommendations: profile personalized shelf
CREATE INDEX idx_recs_profile_score  ON recommendations (profile_id, score DESC);

-- Downloads: active downloads per device
CREATE INDEX idx_downloads_profile   ON downloads (profile_id, expires_at);
CREATE INDEX idx_downloads_device    ON downloads (device_id, expires_at);

```

Query Patterns

Q1 — Homepage: Continue Watching Shelf

```

SELECT
    wp.title_id,
    wp.episode_id,
    wp.progress_seconds,
    wp.duration_seconds,
    t.name,
    t.poster_url,
    e.episode_number,
    e.name AS episode_name,
    s.season_number
FROM watch_progress wp

```

```

JOIN titles t ON t.title_id = wp.title_id
LEFT JOIN episodes e ON e.episode_id = wp.episode_id
LEFT JOIN seasons s ON s.season_id = e.season_id
WHERE wp.profile_id = $1
      AND wp.completed = FALSE
ORDER BY wp.updated_at DESC
LIMIT 10;

```

Q2 — Homepage: Personalized Recommendations Shelf

```

SELECT
    r.title_id, r.score, r.reason,
    t.name, t.content_type, t.poster_url,
    t.avg_user_rating, t.release_year
FROM recommendations r
JOIN titles t ON t.title_id = r.title_id
JOIN title_availability ta
    ON ta.title_id = r.title_id AND ta.country_code = $2
WHERE r.profile_id = $1
      AND t.is_active
ORDER BY r.score DESC
LIMIT 20;

```

Q3 — Title Detail Page

```

WITH cast_data AS (
    SELECT
        tc.title_id,
        json_agg(jsonb_build_object(
            'name', p.name,
            'role', tc.role,
            'character', tc.character_name,
            'photo', p.photo_url
        ) ORDER BY tc.sort_order) AS cast
    FROM title_cast tc
    JOIN people p ON p.person_id = tc.person_id
    WHERE tc.title_id = $1
    GROUP BY tc.title_id
),
genre_data AS (
    SELECT
        tg.title_id,
        array_agg(g.name ORDER BY g.name) AS genres
    FROM title_genres tg
    JOIN genres g ON g.genre_id = tg.genre_id
    WHERE tg.title_id = $1
    GROUP BY tg.title_id
)
SELECT

```

```

t.*,
cd.cast,
gd.genres,
ta.available
FROM titles t
LEFT JOIN cast_data cd ON cd.title_id = t.title_id
LEFT JOIN genre_data gd ON gd.title_id = t.title_id
LEFT JOIN title_availability ta
    ON ta.title_id = t.title_id AND ta.country_code = $2
WHERE t.title_id = $1;

```

Q4 — Series: All Seasons and Episodes

```

SELECT
    s.season_id, s.season_number, s.name AS season_name,
    json_agg(jsonb_build_object(
        'episode_id', e.episode_id,
        'number', e.episode_number,
        'name', e.name,
        'runtime', e.runtime_minutes,
        'still', e.still_url,
        'synopsis', e.synopsis,
        'progress', wp.progress_seconds,
        'duration', wp.duration_seconds
    ) ORDER BY e.episode_number) AS episodes
FROM seasons s
JOIN episodes e ON e.season_id = s.season_id AND e.is_active
LEFT JOIN watch_progress wp
    ON wp.episode_id = e.episode_id AND wp.profile_id = $2
WHERE s.title_id = $1
GROUP BY s.season_id
ORDER BY s.season_number;

```

Q5 — Upsert Watch Progress (frequent write)

```

INSERT INTO watch_progress
    (profile_id, title_id, episode_id, progress_seconds, duration_seconds,
    completed)
VALUES ($1, $2, $3, $4, $5, $6)
ON CONFLICT (profile_id, title_id) DO UPDATE
    SET progress_seconds = EXCLUDED.progress_seconds,
        episode_id       = EXCLUDED.episode_id,
        completed        = EXCLUDED.completed,
        updated_at       = now()
WHERE watch_progress.progress_seconds < EXCLUDED.progress_seconds;
-- Only update if new position is further ahead (prevents backwards scrub
overwriting)

```

Q6 — Full-Text Search

```

SELECT
    t.title_id,
    t.name,
    t.content_type,
    t.release_year,
    t.poster_url,
    t.avg_user_rating,
    ts_rank(t.search_vector, query) AS rank,
    ta.available
FROM titles t,
    to_tsquery('english', $1) query
JOIN title_availability ta ON ta.title_id = t.title_id AND ta.country_code = $2
WHERE t.search_vector @@ query
    AND t.is_active
ORDER BY rank DESC, t.avg_user_rating DESC NULLS LAST
LIMIT 30;

```

Q7 — Browse by Genre with Availability

```

SELECT
    t.title_id, t.name, t.content_type,
    t.avg_user_rating, t.release_year, t.poster_url
FROM titles t
JOIN title_genres tg ON tg.title_id = t.title_id
JOIN title_availability ta ON ta.title_id = t.title_id AND ta.country_code = $2
WHERE tg.genre_id = $1
    AND t.is_active
ORDER BY t.avg_user_rating DESC NULLS LAST, t.release_year DESC
LIMIT 50 OFFSET $3;

```

Q8 — User Rating Insert/Update

```

INSERT INTO user_ratings (profile_id, title_id, rating)
VALUES ($1, $2, $3)
ON CONFLICT (profile_id, title_id) DO UPDATE
    SET rating = EXCLUDED.rating,
        created_at = now();

-- After this, update the aggregate (via trigger or async job)
UPDATE titles
SET avg_user_rating = (
    SELECT AVG(rating) FROM user_ratings WHERE title_id = $2
),
total_ratings = (
    SELECT COUNT(*) FROM user_ratings WHERE title_id = $2
)
WHERE title_id = $2;

```

Q9 — Billing: Monthly Subscription Charge Candidates

```
SELECT
    a.account_id, a.email, a.plan, a.country_code,
    pm.token, pm.provider,
    sp.price_usd_cents
FROM accounts a
JOIN payment_methods pm ON pm.account_id = a.account_id AND pm.is_default
JOIN subscription_plans sp ON sp.plan = a.plan
WHERE a.status = 'active'
    AND NOT EXISTS (
        SELECT 1 FROM billing_periods bp
        WHERE bp.account_id = a.account_id
            AND bp.period_start = DATE_TRUNC('month', CURRENT_DATE)
            AND bp.status = 'paid'
    )
ORDER BY a.account_id
LIMIT 10000; -- Process in batches
```

Q10 — Expiring Downloads Cleanup

```
DELETE FROM downloads
WHERE expires_at < now()
RETURNING profile_id, asset_id, device_id;
```

Scaling Strategy

Read-Heavy Optimization (Core Challenge)

The platform is ~95% reads. The primary scaling lever is aggressive caching:

Layer	What is cached	TTL
CDN	Poster images, trailer thumbnails	30 days
API Gateway cache	Title detail pages	60 seconds
Redis	"Continue Watching" per profile	5 minutes
Redis	Recommendations per profile	1 hour
Redis	Genre shelves per country	5 minutes
PostgreSQL materialized view	title_availability	Refreshed nightly

Phase 1: Single Region (0–50M subscribers)

- Primary + 2 read replicas
- Redis Cluster for watch_progress and recommendations
- Separate analytics replica for reporting

Phase 2: Global Expansion (50M–200M subscribers)

- Deploy per-region PostgreSQL clusters (US, EU, APAC)
- Catalog DB is global (read-only replicas in each region, writes to primary)
- Watch history and billing are region-local
- Licensing materialized view refreshed per region
- watch_history partitioned by month; partitions older than 12 months archived to columnar store (Redshift, BigQuery) for analytics

Phase 3: Planet-Scale (200M+ subscribers)

- Separate databases: CatalogDB, ProfileDB, WatchDB, BillingDB
- watch_progress and recommendations moved to Redis with async persistence
- Watch events streamed via Kafka → consumed by both PostgreSQL (history) and ML pipeline (recommendations)
- ContentDeliveryNetwork handles all video assets; PostgreSQL stores only metadata

Tradeoffs

Decision	Chosen	Alternative	Why
Separate watch_progress table	Denormalized hot table	Query watch_history	"Continue Watching" is the most-read feature; hot table avoids partition scan
Materialized view for licenses	Precomputed availability	Join at query time	Reduces every title query from 3 joins to 1; refreshed nightly
UPSERT for watch_progress	INSERT ... ON CONFLICT	UPDATE always	Handles first-watch and resume in one statement
JSON for recommendations	External table	ML API call at read time	Sub-millisecond reads; recs are batch-computed anyway
Monthly billing partitions	Range by date	No partition	Old billing data never queried; partition pruning on account history queries

Interview Discussion Points

1. **How do you handle the "Continue Watching" feature at scale?** The watch_progress table is a denormalized materialized summary. Every video player tick (every 30 seconds) does an UPSERT on this small table. At 100M DAU × 1 event/30s = ~3.3M writes/min — Redis absorbs this, and we async-flush to PostgreSQL every 5 minutes.
2. **How does content licensing work?** Each title has region-specific licenses. Rather than joining licenses on every title query, we maintain a materialized view title_availability that precomputes (title_id, country_code) availability. This view is refreshed once per night when license expirations are processed.
3. **Why partition watch_history by month?** A viewer who watched 500 titles in 5 years generates 500 rows. At 1.25B profiles, that is 625B rows. 98% of queries are for "recent" watches (last 90 days). Monthly partitions let PostgreSQL skip all but the last 3 partitions for any user query.

4. **How do recommendations work at the DB level?** The ML model runs in a separate pipeline, computes a ranked list of ~50 titles per profile, and bulk-inserts into the `recommendations` table. The DB serves this pre-computed list; no ML inference happens at query time.
-

Common Interview Follow-ups

Q: How would you implement "because you watched X, you might like Y"? A: The recommendations table includes a `reason` column. When the ML pipeline generates the recommendations, it populates the reason (e.g., `'because_you_watched:title_id_xxx'`). The API reads this and hydrates the UI label.

Q: How do you handle content that is available in some countries but not others? A: Every API call includes the user's `country_code`. All title queries join against the `title_availability` materialized view filtered by `country_code`. This is cheap — the view has a unique index on `(title_id, country_code)`.

Q: How would you implement a "Top 10 in Your Country" feature? A: Maintain a separate table `trending_titles(country_code, title_id, rank, updated_at)` refreshed hourly from `watch_history` aggregation. Never compute this at query time.

Performance Considerations

- **watch_progress is the hottest table.** Ensure `fillfactor=70` to leave room for HOT updates (Heap-Only Tuple), reducing index maintenance overhead on every UPSERT.
 - **Avoid COUNT(*) on titles per genre** — maintain `genre_title_count` in a summary table updated by trigger.
 - **Index-only scans** on `title_availability(title_id, country_code)` avoid heap fetches entirely — keep VACUUM aggressive.
 - **Batch watch_history writes:** Buffer 30-second heartbeat events in Redis, flush to PostgreSQL in bulk every 5 minutes via background job. Reduces write amplification by 10x.
 - **Partition pruning:** Always include `started_at >= $date` in `watch_history` queries to enable partition pruning.
-

Cross-References

- See `06_messaging_system.md` for LISTEN/NOTIFY patterns applicable to real-time notifications
- See `21_System_Design/07_analytics_platform_design.md` for watch event analytics pipeline
- See `21_System_Design/08_system_design_framework.md` for interview framework
- PostgreSQL materialized views: Chapter 7 of this guide
- Table partitioning: Chapter 5 of this guide